

```
In [85]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from imblearn.over_sampling import SMOTE
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
classification_report, precision_score, recall_score, accuracy_score
```

```
In [86]: df = pd.read_csv('nba_naive.csv')
```

```
In [87]: target_column = 'target_5yrs'
feature_columns = [col for col in df.columns if col != target_column]

X = df[feature_columns]
y = df[target_column]

print(f"Features (X): {list(X.columns)}")
print(f"Target (y): {y.name}")
```

```
Features (X): ['fg', '3p', 'ft', 'reb', 'ast', 'stl', 'blk',
'tov', 'total_points', 'efficiency']
Target (y): target_5yrs
```

```
In [88]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (938, 10)
X_test shape: (402, 10)
y_train shape: (938,)
y_test shape: (402,)
```

```
In [89]: print("Original Target Class Distribution:")
print(y.value_counts(normalize=True))

print("\nTraining Set Target Class Distribution:")
print(y_train.value_counts(normalize=True))

print("\nTest Set Target Class Distribution:")
print(y_test.value_counts(normalize=True))

plt.figure(figsize=(12, 5))

plt.subplot(1, 3, 1)
sns.countplot(x=y, palette='viridis')
plt.title('Original Target Distribution')

plt.subplot(1, 3, 2)
sns.countplot(x=y_train, palette='viridis')
plt.title('Training Set Target Distribution')

plt.subplot(1, 3, 3)
sns.countplot(x=y_test, palette='viridis')
plt.title('Test Set Target Distribution')

plt.tight_layout()
plt.show()
```

Original Target Class Distribution:

target_5yrs

1 0.620149

0 0.379851

Name: proportion, dtype: float64

Training Set Target Class Distribution:

target_5yrs

1 0.620469

0 0.379531

Name: proportion, dtype: float64

Test Set Target Class Distribution:

target_5yrs

1 0.619403

0 0.380597

Name: proportion, dtype: float64

C:\Users\LENOVO

THINKPAD\AppData\Local\Temp\ipykernel_19560\592465376.py:13:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

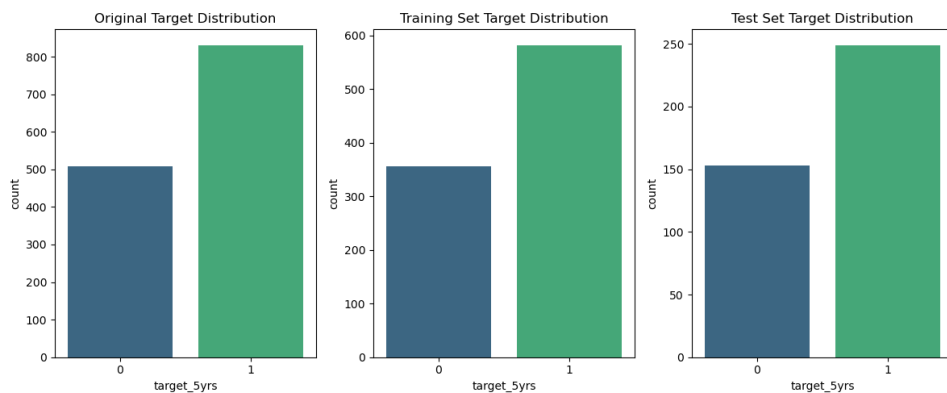
```
sns.countplot(x=y, palette='viridis')
C:\Users\LENOVO
THINKPAD\AppData\Local\Temp\ipykernel_19560\592465376.py:17:
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x=y_train, palette='viridis')
C:\Users\LENOVO
THINKPAD\AppData\Local\Temp\ipykernel_19560\592465376.py:21:
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x=y_test, palette='viridis')
```



```
In [90]: gnb = GaussianNB()
gnb.fit(X_train, y_train)

print("Gaussian Naive Bayes model trained successfully.")
```

Gaussian Naive Bayes model trained successfully.

```
In [91]: y_pred = gnb.predict(X_test)

print("Predictions on the test set made.")
```

Predictions on the test set made.

```
In [92]: print("Classification Report:")
print(classification_report(y_test, y_pred))

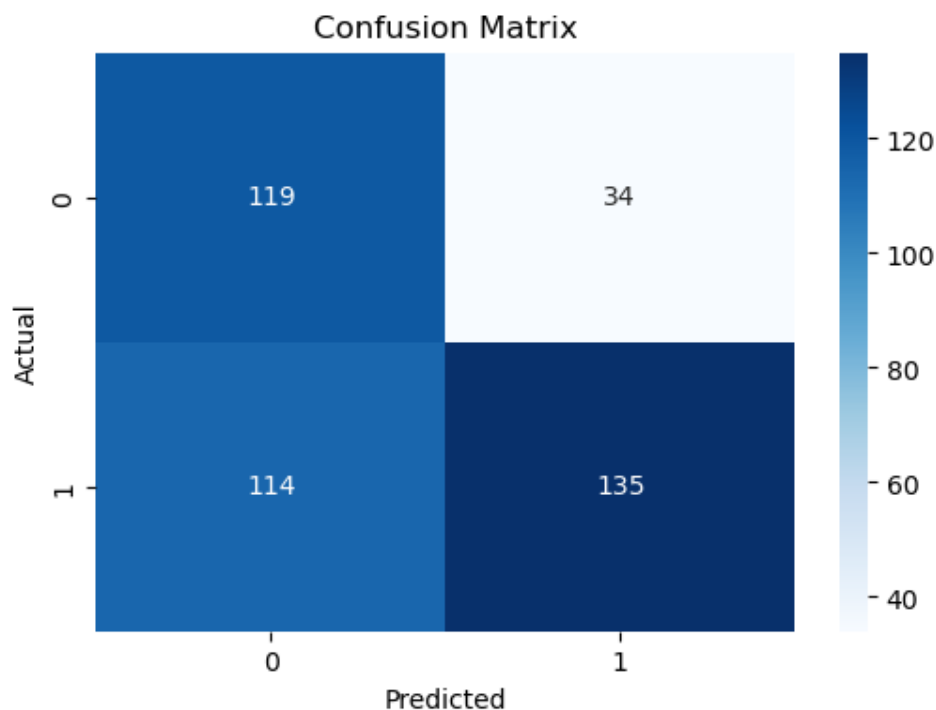
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy Score: {accuracy:.4f}")

conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=gnb.classes_, yticklabels=gnb.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Classification Report:

	precision	recall	f1-score	support
0	0.51	0.78	0.62	153
1	0.80	0.54	0.65	249
accuracy			0.63	402
macro avg	0.65	0.66	0.63	402
weighted avg	0.69	0.63	0.63	402

Accuracy Score: 0.6318



```
In [93]: from sklearn.inspection import permutation_importance

result = permutation_importance(gnb, X_test, y_test, n_repeats=10,
                               random_state=42, n_jobs=-1)

perm_importance_df = pd.DataFrame({
    'feature': X_test.columns,
    'importance_mean': result.importances_mean,
    'importance_std': result.importances_std
})

perm_importance_df =
perm_importance_df.sort_values(by='importance_mean', ascending=False)
display(perm_importance_df)

fig = plt.figure(figsize=(12, 6))
sns.barplot(x='importance_mean', y='feature', data=perm_importance_df,
            palette='viridis')
plt.title('Permutation Feature Importance for Gaussian Naive Bayes')
plt.xlabel('Mean Decrease in Accuracy (Importance)')
plt.ylabel('Feature')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

	feature	importance_mean	importance_std
8	total_points	0.031343	0.015701
6	blk	0.028358	0.010025
0	fg	0.015423	0.005651
3	reb	0.013682	0.009258
9	efficiency	0.006965	0.006073
1	3p	0.000249	0.002347
5	stl	-0.003483	0.009900
2	ft	-0.003980	0.003374
4	ast	-0.004229	0.007128
7	tov	-0.013682	0.008041

C:\Users\LENOVO

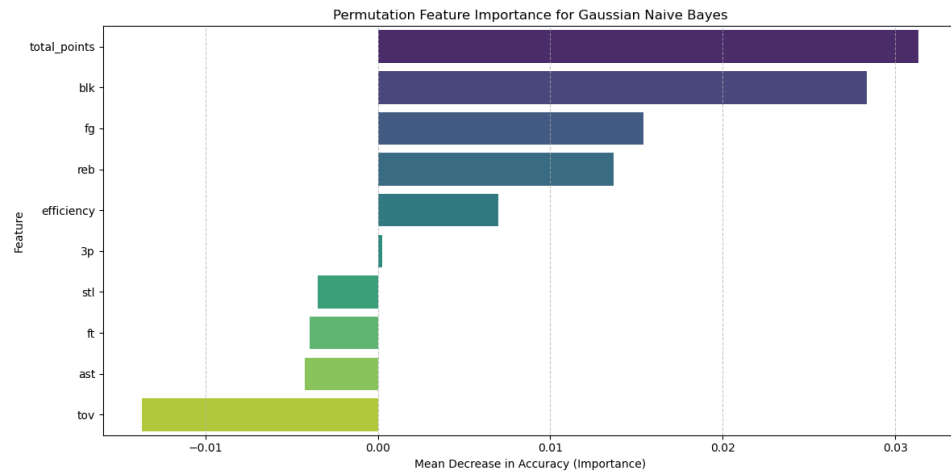
THINKPAD\AppData\Local\Temp\ipykernel_19560\4074266905.py:15:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and
will be removed in v0.14.0. Assign the `y` variable to `hue`

and set `legend=False` for the same effect.

```
sns.barplot(x='importance_mean', y='feature',  
data=perm_importance_df, palette='viridis')
```



When to Trust (and Question) Model Outputs: A Stakeholder Summary

We've developed and evaluated a **Gaussian Naive Bayes (GNB)** model to predict player longevity (`target_5yrs`) in the NBA based on key performance statistics. Here's a breakdown of when you can place confidence in its predictions and when it's prudent to ask further questions.

When to Trust the Model Outputs:

1. **Overall Predictive Accuracy (Moderate):** The model achieved an **overall accuracy of approximately 63%**. This means it correctly classified about 63 out of 100 players regarding their longevity. For quick, high-level screening or as a baseline, this can provide useful insights.

2. **Identifying Players *Not* Likely to Last (Class 0):** The model shows good **recall for Class 0 (players not likely to last 5 years)** at **78%**. This means that when a player was *actually* not going to last, the model correctly identified them 78% of the time. This makes the model reasonably reliable for identifying potential 'busts' or players with short careers.

3. **Key Influential Features Align with Intuition:** The **Permutation Feature Importance** highlighted `total_points`, `blk` (blocks), `fg` (field goal percentage), and `reb` (rebounds) as the most impactful features. This intuitively makes sense, as these are fundamental aspects of a player's on-court contribution and often correlate with success and continued play. If these core stats are strong, the model is more likely to predict longevity.

When to Question the Model Outputs:

1. **Risk of Misclassifying 'Long-Lasting' Players (Low Recall for Class 1):** The model's **recall for Class 1 (players likely to last 5 years)** is **54%**. This means that nearly half of the players who *actually* lasted 5 years or more were incorrectly predicted as *not* lasting. Relying solely on this model to identify future stars could lead to missing out on talented individuals.

2. Interpreting 'Likely to Last' Predictions (Moderate Precision for Class 1): While the precision for Class 1 is **80%** (meaning 80% of players *predicted* to last actually did), the low recall means many true positives are missed. If a player is predicted to last, there's a good chance they will, but the model isn't catching everyone who will succeed.

3. Violation of Model Assumptions (Feature Independence): The **Gaussian Naive Bayes model assumes that all input features are independent of each other**, given the target variable. In sports, this is rarely true (e.g., higher field goal percentage directly impacts total points). While GNB can still perform adequately, this violation means the probability estimates might be overconfident, and the model might not capture complex interdependencies crucial for player success. This suggests that the model is a simplified view of reality.

4. Features with Negative Importance: Some features like `tov` (turnovers), `ast` (assists), `ft` (free throws), and `stl` (steals) showed **negative permutation importance**. This is an interesting outcome; it doesn't necessarily mean these features are irrelevant in reality, but rather that within this specific GNB model, shuffling them didn't significantly harm (and sometimes slightly improved) performance. This can happen with correlated features or in less robust models. It suggests that the model might not be leveraging these features effectively, or their relationships are complex and not well-captured by the GNB's simplistic assumptions.

Recommendation:

- * **Use the model for quick, initial screening**, particularly for flagging players who are statistically less likely to have long careers (Class 0).
- * **Exercise caution when making definitive positive predictions (Class 1)**, as the model misses many actual long-lasting players. Combine its insights with domain expertise and other scouting methods.
- * **Consider this model as a baseline.** Its simplicity makes it fast, but for more nuanced and accurate predictions, especially of long-term success, a more sophisticated model that can handle feature dependencies might be warranted (e.g., Logistic Regression, Tree-based models).
- * **Review the influential features.** While `total_points`, `blk`, `fg`, and `reb` are strong indicators, understand that the model might not be fully capturing the value of other statistics due to its inherent assumptions.

Impact on Model Validity in Sports Contexts (e.g., NBA Player Data)

In sports data, especially for player statistics like those in `nba_naive.csv`, this independence assumption is almost always violated. Let's consider some of the features we used:

- `fg` (Field Goal Percentage) and `total_points`: A higher field goal percentage directly contributes to more total points (assuming similar shot attempts). These are clearly not independent.
- `reb` (Rebounds), `ast` (Assists), `stl` (Steals), `blk` (Blocks), `tov` (Turnovers): These are all highly interconnected aspects of a player's overall game performance. For example, a player with many assists (`ast`) is often a primary ball-handler, which might correlate with a higher number of turnovers (`tov`). A player with a high number of blocks (`blk`) is likely a good defender, which might also influence their rebound numbers (`reb`).
- `efficiency`: This is often a composite metric derived from several other statistics (like points, rebounds, assists, steals, blocks, and turnovers). If it's included as a feature alongside its components, it introduces strong multicollinearity, directly violating the independence assumption.

Consequences of Violated Independence Assumption:

- Suboptimal Performance: While Naive Bayes can still perform surprisingly well even when the independence assumption is violated (especially if the relationships between features are consistent across classes), it's unlikely to be the optimal model. The model might not fully capture the complex interplay between different player statistics, leading to a less accurate representation of player potential.
- Poor Probability Estimates: The most significant impact of violated independence is on the predicted probabilities. The probability estimates $P(y | x)$ might be inaccurate or overly confident because the model incorrectly assumes that each piece of evidence (each feature) is independent, thus double-counting the strength of related evidence.
- Interpretability Issues (for feature importance): While Naive Bayes doesn't directly provide traditional feature importance in the same way as, say, a tree-based model, attempting to infer feature contributions might be misleading because the model treats them as separate entities, ignoring their co-dependencies.

Why Use Gaussian Naive Bayes Then?

Despite these limitations, Gaussian Naive Bayes remains popular for several reasons:

- Simplicity and Speed: It's very fast to train and predict, even on large datasets, and requires minimal tuning.
- Good Baseline: It often serves as an excellent baseline model against which more

complex models can be compared.

- Robustness: Sometimes, even with strong violations of independence, it can yield surprisingly good classification performance, especially if the features are only mildly correlated or if the relationships are similar across classes.

Conclusion:

In the context of sports data, particularly with features derived from game statistics, the independence assumption of Gaussian Naive Bayes is almost certainly violated. This doesn't necessarily mean the model is useless, as it can still provide reasonable classification accuracy and serve as a quick baseline. However, for more accurate probability estimates or to capture the nuanced relationships between features, other models that do not make such strict independence assumptions (e.g., Logistic Regression, Support Vector Machines, Tree-based models like Random Forests or Gradient Boosting) would generally be more appropriate and valid.

Actionable Recommendations for Integrating the Model into Scouting Workflows

Given the performance characteristics and limitations of our Gaussian Naive Bayes model, here are actionable recommendations for how it can be integrated into existing NBA scouting workflows:

1. Initial Screening for 'High-Risk' Prospects (Focus on Class 0 Recall)

* **Recommendation:** Use the model as an **early warning system** to flag players who are statistically less likely to play for 5+ years. Given its **78% recall for Class 0 (players not lasting 5 years)**, it's reasonably effective at identifying prospects with underlying stats that historically don't translate to longevity.

* **Action:** Automatically apply the model to draft combine data or early season statistics for young players. Any player predicted by the model to be in Class 0 (not lasting 5 years) should trigger a **deeper dive** by human scouts.

* **Benefit:** Helps prioritize scouting resources by narrowing down the pool of players requiring intensive analysis, focusing on those most likely to disappoint expectations based on their statistical profile.

2. Complementary Tool, Not a Replacement (Acknowledge Low Class 1 Recall)

Recommendation: *Emphasize that the model is a **complementary tool** to human scouting, not a replacement. Its **54% recall for Class 1 (players lasting 5 years)** means it will miss nearly half of the players who do* go on to have long careers.*

* **Action:** Do not exclusively rely on the model for identifying 'long-term' prospects. If a player is predicted by the model to be in Class 0 but human scouts have identified them as high-potential, **trust the human intuition and qualitative assessment** due to the model's known limitations in this area.

* **Benefit:** Prevents overlooking potential stars who might not fit the model's statistical archetype but possess other qualities (e.g., leadership, work ethic, coachability) that lead to longevity.

3. Validate and Explain Predictions Using Key Features (Leverage Feature Importance)

* **Recommendation:** When the model makes a prediction, especially for Class 0, review the player's performance in the **most influential features**: `total_points`, `blk`, `fg`, and `reb`.

* **Action:** For a player flagged as 'high-risk' (Class 0), a scouting report could highlight their lower numbers in these critical areas, providing a quantitative basis for the model's concerns. Conversely, if a player predicted to last (Class 1) excels in these areas, it reinforces the model's decision.

***Benefit:** Provides a more transparent and data-driven explanation for model outputs, helping scouts understand why the model made a particular prediction, rather than just what* it predicted.*

4. Set Realistic Expectations (Understand Model Limitations)

* **Recommendation:** Educate stakeholders on the model's assumptions and limitations, particularly the **violation of feature independence** inherent in Naive Bayes for sports statistics.

* **Action:** Communicate that the model provides a simplified, statistical lens on player longevity and is best suited for broad categorization rather than precise predictions for individual players. Avoid presenting its probability outputs as absolute truths.

* **Benefit:** Manages expectations and builds trust by being transparent about what the model can and cannot reliably do, avoiding misinterpretation of its outputs.

5. Iterative Improvement and Exploration of Advanced Models (Continuous Development)

* **Recommendation:** View this GNB model as a **baseline**. Encourage future iterations to explore more sophisticated machine learning models that can better handle feature interdependencies.

* **Action:** Plan for future development to experiment with models like Logistic Regression, Support Vector Machines, Random Forests, or Gradient Boosting, which might offer improved accuracy and better capture the complex relationships within NBA player statistics.

* **Benefit:** Ensures that the analytics department is continuously seeking to improve predictive capabilities and adapt to new methodologies, providing increasingly valuable tools for scouting.

Exported with [runcell](https://www.runcell.dev) — convert notebooks to HTML or PDF anytime at [runcell.dev](https://www.runcell.dev).